

AI Repository Documentation Report

Repository ID: 897cbc83-2d0c-4533-9227-0d11269d69e7

Generated by Repository AI Architect

Project Overview

This project is designed to enhance a chess game by integrating a locally-hosted, open-source Language Model (LLM) to generate dynamic and engaging narrative descriptions for each move. The LLM acts as the central component, deciding both the narration text and the corresponding visual effect for the frontend animations. In cases where the LLM is unreachable or returns unparsable data, the system gracefully falls back to predefined, dramatic narrations and effects to ensure the game remains uninterrupted.

Architecture

The architecture of this project revolves around a single function ``get_narration`` which interacts with the LLM through HTTP POST requests. It constructs a prompt based on the move information, sends it to the LLM, and processes the response. If the interaction with the LLM fails, it uses fallback functions ``_fallback_narration`` and ``_fallback_effect`` to provide default narrations and effects.

Key Components

- **LLM Interaction**: The core functionality lies in communicating with the LLM hosted locally via Ollama.
- **Prompt Construction**: The ``build_prompt`` function formats the move information into a structured prompt for the LLM.
- **Response Parsing**: The response from the LLM is parsed to extract the narration and effect.
- **Fallback Mechanism**: Ensures that the game can continue with default narrations and effects if the LLM interaction fails.

Main Functions

- **``get_narration(move_info, timeout=8)``**: This function takes move information and generates a narration and effect either from the LLM or from fallback mechanisms.
- **``build_prompt(move_info)``**: Constructs the prompt string to be sent to the LLM.
- **``_fallback_narration(move_info)``**: Provides a default narration based on the type of move.
- **``_fallback_effect(move_info)``**: Selects a default effect from a predefined list based on the category of the move.

- `**`_category(move_info)`**`: Determines the category of the move (checkmate, capture, check, castle, normal).

Folder Structure

Given the limited context, the folder structure is not explicitly detailed. However, based on the components involved, a typical structure might look like this:

...

chess_rpg/

■

■■■ src/

■ ■■■ __init__.py

■ ■■■ main.py

■ ■■■ utils.py

■

■■■ tests/

■ ■■■ test_main.py

■ ■■■ test_utils.py

■

■■■ README.md

■■■ requirements.txt

...

- `**`src/`**`: Contains the source code files.
- `**`tests/`**`: Contains test files for the application.
- `**`README.md`**`: Provides an overview and setup instructions for the project.
- `**`requirements.txt`**`: Lists all the dependencies required for the project.

Data Flow

1. **Input**: Move information (``move_info``) is received.
2. **Prompt Construction**: The ``build_prompt`` function formats the move information into a prompt.
3. **LLM Communication**: The formatted prompt is sent to the LLM via a POST request.
4. **Response Handling**: The response is parsed to extract the narration and effect.
5. **Validation**: Checks if the extracted narration and effect are valid.
6. **Fallback**: If the LLM response is invalid or the communication fails, the system uses ``_fallback_narration`` and ``_fallback_effect``.
7. **Output**: Returns a dictionary containing the narration, effect, and source of the information.

Setup Instructions

To set up and run this project, follow these steps:

1. **Install Dependencies**:

```
```bash
```

```
pip install -r requirements.txt
```

```
```
```

2. **Locally Host the LLM**:

- Ensure you have Ollama installed and running on your machine.
- Configure the LLM to use the desired model (e.g., Llama 3, Mistral, Qwen).

3. **Configure Environment Variables**:

- Set ``OLLAMA_URL`` to the URL where your LLM is hosted.
- Set ``MODEL_NAME`` to the name of the model you wish to use.

4. **Run the Application**:

- Execute the main script or integrate it into your chess game application.

```
```bash
```

```
python src/main.py
```

```
```
```

Future Improvements

- **Enhanced Fallbacks**: Improve the fallback narrations and effects to be more context-aware.
- **Model Training**: Train the LLM specifically for generating chess-related narrations and effects.
- **User Customization**: Allow users to customize the LLM prompts and fallback options.
- **Error Logging**: Implement comprehensive logging for errors encountered during LLM interactions.
- **Performance Optimization**: Optimize the prompt construction and response parsing for better performance.

This documentation provides a clear overview of the project, its architecture, key components, and setup instructions. Future improvements are suggested to enhance the functionality and user experience of the application